

Abstract Classes, Interfaces & Pure Virtual Functions

1. The Problem: Abstraction Needs Enforcement

Recall from OOP Fundamentals: **abstraction** hides implementation, exposing only functionality. But a plain base class with regular (non-pure) virtual methods doesn't *force* derived classes to actually provide meaningful implementations — a derived class could simply not override anything and inherit a possibly-nonsensical default. Abstract classes and interfaces exist specifically to make certain methods **mandatory** to implement, and in the case of a fully abstract type, to prevent the type from ever being instantiated on its own at all — since a "generic Shape" or "generic Animal" often shouldn't be creatable as a standalone object in the first place.

2. Pure Virtual Functions & Abstract Classes (C++)

A **pure virtual function** is a virtual function with **no implementation** in the class where it's declared, marked with `= 0`.

```
class AbstractClass {
public:
    virtual void display() = 0;    // PURE virtual function – no body
};
```

A class containing at least one pure virtual function automatically becomes an *abstract class* — and abstract classes cannot be instantiated directly.

```
class Shapes {
public:
    virtual float area() = 0;      // pure virtual – Shapes is now ABSTRACT
};

Shapes s;                        // COMPILE ERROR – cannot instantiate an abstract class
Shapes* s = new Shapes();        // COMPILE ERROR – same reason

class Square : public Shapes {
public:
    float area() override { return side * side; }    // MUST override, or Square is ALSO abstract
};

Square sq;                        // fine – Square provides a real implementation for every pure virtual function
```

Why `= 0` specifically forces derived classes to implement it: it tells the compiler "this function has no body at all in this class" — any derived class that doesn't provide its own implementation **inherits the same unimplemented obligation**, and is therefore *also* abstract (and also can't be

instantiated) — the obligation to implement only disappears once some class in the chain actually provides a real body.

A class can mix pure virtual and regular methods — an abstract class can absolutely have fully-implemented, concrete methods alongside its pure virtual ones; it only takes **one** pure virtual function (without an override) to make the whole class abstract.

3. Interfaces (Java) — the language-level equivalent

Java has no direct `= 0` syntax — instead, it has a dedicated language construct, the **interface**: a pure contract specifying **what** a class must do, not **how**.

```
// Any car CAN drive itself if it implements this interface ("CAN-DO" relationship)
interface SelfDrivable {
    void accelerate(double targetSpeedKmh);
    void brake(double targetSpeedKmh);
    void steerTo(double latitude, double longitude);
    boolean isNavigating();
}
```

- Traditionally: **no** instance variables, **no** method bodies — all methods are implicitly `public abstract`, all fields implicitly `public static final` (constants).
- A class **implements** an interface using the `implements` keyword, and must provide a concrete body for **every** declared method — or the implementing class itself must also be declared `abstract`.

```
class NexonEV extends Car implements SelfDrivable {
    private boolean navigating = false;
    NexonEV(double price) { super("Tata", price); }

    @Override public void accelerate(double targetSpeedKmh) {
        System.out.println("Nexon EV accelerating to " + targetSpeedKmh + " km/h");
    }
    @Override public void brake(double targetSpeedKmh) { /* ... */ }
    @Override public void steerTo(double lat, double lon) { navigating = true; /* ... */ }
    @Override public boolean isNavigating() { return navigating; }
}
```

Why interfaces matter — the "CAN-DO" relationship: a class can implement **multiple** interfaces (recall from Inheritance: Java forbids multiple *class* inheritance, but interfaces sidestep the diamond problem, since — traditionally — they carry no state/implementation to conflict over). The real power: **calling code never needs to know the concrete type** — only that it fulfills the interface.

```
// The autonomous module works with ANY SelfDrivable car
class AutonomousModule {
    void navigate(SelfDrivable car, double lat, double lon) {
        car.steerTo(lat, lon);
    }
}
```

`AutonomousModule` works with a `NexonEV`, or any future car model — **as long as it implements `SelfDrivable`**. A brand-new car model can be added later, and `AutonomousModule` needs **zero changes**.

— exactly the same "extensibility without modification" payoff seen in the Polymorphism topic's `Shapes[]` example, now expressed through a pure contract rather than a shared concrete base class.

4. Abstract Class vs. Interface — the comparison every interview asks for

	Abstract Class	Interface
Can have concrete (implemented) methods?	Yes, freely	Traditionally no (Java 8+ allows <code>default</code> / <code>static</code> methods — see §5)
Can have instance fields/state?	Yes	No (only <code>public static final</code> constants)
Multiple inheritance	A class can extend only one abstract class	A class can implement many interfaces
Constructors	Yes (called via the derived class's constructor chain)	No
Represents	An "is-a" relationship, often with shared implementation to reuse	A "can-do" capability/contract, with no shared state
When to use	Related classes sharing significant common code/state, with some behavior that must still be specialized	Unrelated classes that should all support the same capability , with no shared implementation to offer

Interview soundbite: "The real design question isn't 'abstract class or interface' as syntax trivia — it's 'do these types share actual state and implementation I want to reuse (→ abstract class), or do they just need to promise the same capability with otherwise unrelated implementations (→ interface)?' A `Car` hierarchy sharing `price`, `brand`, and common logic is naturally an abstract class; `SelfDrivable` being implementable by a car, a drone, or a robot vacuum — completely unrelated types — is naturally an interface."

5. Default & Static Methods in Interfaces (Java 8+) — a genuine evolution worth knowing

Modern Java interfaces **can** have method bodies, via two special forms:

```

interface SelfDrivable {
    void steerTo(double lat, double lon);

    default void honk() { // DEFAULT method – has a body
        System.out.println("Beep beep!");
    }
    static boolean isValidCoordinate(double lat) { // STATIC method – has a body, called on the interface itself
        return lat >= -90 && lat <= 90;
    }
}

```

- **default methods** — give interfaces an optional, inheritable implementation, so existing implementing classes **don't break** when a new method is added to the interface later (a real historical motivation — this was introduced specifically to let the Java standard library add new methods to widely-implemented interfaces like `Collection` without breaking every existing implementation in the world).
- **static methods** — utility methods that logically belong with the interface but don't operate on any specific implementing instance.

Interaction with the diamond problem (recall Inheritance topic §4): if a class implements two interfaces that both provide a **conflicting default** method with the same signature, Java does **not** silently pick one — it's a **compile error**, forcing the implementing class to explicitly override the method and resolve the conflict itself (potentially calling one or both via `InterfaceName.super.method()`).

6. Why Abstract Types Matter for Design (not just syntax)

- **Preventing meaningless instantiation** — a generic `Shape` with no defined `area()` genuinely shouldn't be creatable; making it abstract enforces this at compile time rather than relying on discipline/documentation.
- **Enforcing a contract across a whole family of subtypes** — every concrete subclass is *guaranteed*, by the compiler, to provide the required behavior — there's no way to "forget" to implement a pure virtual function/interface method and have it silently compile with broken default behavior.
- **Enabling the payoff from Polymorphism** — abstract base classes/interfaces are exactly what makes the "write code against the general type, work with any specific subtype, including future ones" pattern possible and safe.

Interview Questions With Answers

Q1. What specifically does `= 0` mean when appended to a virtual function declaration in C++, and what are its two consequences?

Answer: `= 0` marks a virtual function as "pure" — meaning it has no implementation at all in the class where it's declared. This has two consequences: first, the class containing it automatically becomes an abstract class, which cannot be instantiated directly; second, any derived class that doesn't provide its own implementation of that function inherits the same unimplemented obligation and is therefore also abstract — the requirement only goes away once some class in the inheritance chain actually provides a real implementation.

Q2. Can an abstract class in C++ have any fully-implemented, concrete methods, or must every method be pure virtual?

Answer: An abstract class can freely mix concrete (fully implemented) methods alongside pure virtual ones — it only takes a single unoverridden pure virtual function to make the entire class abstract. This is actually very common in practice: a base class might provide substantial shared, reusable implementation for most of its methods, while leaving just one or two specific behaviors as pure virtual, requiring each concrete subclass to specialize exactly those.

Q3. Traditionally, why can't a Java interface have instance fields the way a class or abstract class can?

Answer: An interface is meant to be a pure contract describing capabilities/behavior — "what a class must be able to do" — without carrying any of its own state at all. Any field declared in an interface is implicitly `public static final` (a constant shared across all implementers, not per-instance state), reinforcing that interfaces traditionally describe behavior only, leaving all actual state to be defined by whatever concrete class implements the interface — this is also part of why interfaces don't suffer the diamond problem the way multiple inheritance of classes-with-state would.

Q4. Give a concrete example of when you'd choose an abstract class over an interface, and explain your reasoning.

Answer: A `Car` hierarchy (e.g., `PetrolCar`, `ElectricCar`, all extending an abstract `Car` base) is a good fit for an abstract class, because these types genuinely share substantial common state (price, brand, model) and potentially common implemented behavior (e.g., a shared `getPrice()`), while still needing some behavior specialized per subtype (e.g., `fuelUp()` behaving differently for petrol vs. electric). An interface wouldn't fit as well here because there's no natural way to share that common state and implementation through an interface — you'd end up duplicating it across every implementing class. The reasoning: choose an abstract class when there's real shared state/implementation to reuse across an "is-a" family; choose an interface when you just need to guarantee a "can-do" capability across otherwise-unrelated types.

Q5. Why were `default` methods added to Java interfaces in Java 8, and what specific problem did they solve?

Answer: Before Java 8, adding any new method to a widely-used interface (like something in the standard Collections library, implemented by countless classes across the ecosystem) would break every single existing class that implements that interface, since Java requires implementing classes to provide a body for every interface method — an existing class simply wouldn't compile anymore the moment the interface it implements gains a new required method. Default methods solve this by letting an interface provide a fallback implementation directly, so existing implementing classes automatically inherit that default behavior for the new method without needing any changes — new interface methods can be added without breaking already-written, already-compiled implementing code.

Q6. What happens if a class implements two interfaces that each provide a conflicting default method with the same signature? Why does Java handle it this way rather than silently picking one?

Answer: Java refuses to compile until the implementing class explicitly overrides the conflicting method itself, resolving the ambiguity manually (optionally delegating to one or both original implementations via `InterfaceName.super.method()`). Java handles it this way rather than silently picking one of the two implementations because silently choosing would be exactly the kind of ambiguous, easily-wrong behavior that plagued multiple inheritance of implementation in other languages (recall the diamond problem) — by forcing an explicit resolution, Java guarantees the implementing class's author has consciously decided what the correct combined behavior should be, rather than depending on some arbitrary, easily-misunderstood tie-breaking rule.

Q7. Why does a class implementing an interface but leaving one required method unimplemented still fail to compile, even if that class is never actually instantiated in a way that would call the missing method?

Answer: The compiler's guarantee is at the *type* level, not based on runtime usage patterns — if a class claims (via `implements`) to fulfill an interface's contract, every caller holding a reference of that interface type is entitled to assume every declared method is safely callable, regardless of how the class actually ends up being used in practice. If the compiler allowed an incomplete implementation to compile as a concrete (non-abstract) class, it would be silently breaking that universal guarantee — some future caller, unaware of the specific gap, could call the missing method and encounter completely undefined/broken behavior. The only way to have an incomplete implementation compile is to explicitly mark the class itself `abstract`, which honestly communicates "this type is not yet fully usable on its own" rather than pretending it fully satisfies the contract.

Q8. Scenario: You're designing a system with `Duck`, `RemoteControlDuck` (a toy), and `RobotDuck` — all of which should be able to `quack()`, but only real `Duck` and `RobotDuck` should be able to `fly()`, while `RemoteControlDuck` cannot. A colleague suggests a single abstract class `AbstractDuck` with both `quack()` and `fly()` as abstract methods, requiring

`RemoteControlDuck` to override `fly()` with an empty or exception-throwing body. What's the problem with this design, and how would interfaces offer a cleaner solution?

Answer: The problem is that forcing `RemoteControlDuck` to implement `fly()` with an empty body or an exception violates the *meaning* of the contract — any code holding an `AbstractDuck` reference and calling `fly()` reasonably expects the duck to actually attempt flying, but for `RemoteControlDuck` this either silently does nothing (surprising, easy to miss) or throws at runtime (a failure that could have been caught much earlier, at compile time, with better design). This is a classic violation of the idea that a subtype should be fully substitutable for its supertype without surprising behavior (closely related to the Liskov Substitution Principle in the SOLID topic). A cleaner solution: separate the two "can-do" capabilities into their own interfaces — `interface Quackable { void quack(); }` and `interface Flyable { void fly(); }` — then `Duck` and `RobotDuck` implement both `Quackable` and `Flyable`, while `RemoteControlDuck` implements only `Quackable`. Now the type system itself accurately reflects reality: it's simply impossible to call `.fly()` on something declared only as `Quackable`, since `RemoteControlDuck` was never claimed to support that capability in the first place — no empty/exception-throwing override needed at all.